

Speckle Pattern

Dissertation submitted for the requirement for the

Degree of

Master of science

In

Physics

Submitted by

Aditya Pratap Singh

Roll No. 237601

Under the supervision of

Dr. Devendra Singh



Department of Physics

School of Physical and Decision Science

Babasaheb Bhimrao Ambedkar University

Lucknow-226025, U.P, (India)

2025

CERTIFICATE

This is to certify that **Mr. ADITYA PRATAP SINGH** has worked under my guidance and the report submitted in the project entitled “**Speckle Pattern**” is suitable for the partial fulfillment of the requirements of the award of the degree of Master of Science in Physics.

Dr. Devendra Singh

School of Physical and Decision Science
Babasaheb Bhimrao Ambedkar University
Lucknow-226025 U.P, India

Dr. Bal Chandra Yadav

(Head Department of Physics)
School of Physical and Decision Science
Babasaheb Bhimrao Ambedkar University
Lucknow – 226025, U.P., India

DECLARATION

I hereby declare that the dissertation entitled “*Speckle Pattern*” submitted in partial fulfillment of the requirements for the award of the degree of **Master of Science in Physics** to the Department of Physics, School of Physical and Decision Science, Babasaheb Bhimrao Ambedkar University, Lucknow, is a Bonafide record of original work carried out by me under the supervision of **Dr. Devendra Singh**, Assistant Professor, Department of Physics, Babasaheb Bhimrao Ambedkar University, Lucknow. It is essentially free from all kinds of plagiarism.

I further declare that this work has not been submitted anywhere else for the award of any other degree. I understand that I shall be responsible for any kind of violation.

Date: -.....

Aditya Pratap Singh

Enrollment No.-074/23

Roll No. – 237601

Department of Physics

Babasaheb Bhimrao Ambedkar University

Lucknow – 226025

Acknowledgement

I would like to express my sincere gratitude to **Dr. Devendra Singh** for his invaluable guidance, encouragement, and support throughout the course of this project on “**Speckle Pattern**”. His expertise and insights have been instrumental in shaping the direction of this work and enhancing my understanding of the subject.

I am thankful to other faculties **Prof. B. C. Yadav, Prof. Devesh Kumar, Late Dr. Ramesh Chandra, Dr. Anil Kumar Yadav, Dr. Khem B. Thapa, Dr. Ajeet Kumar Maurya and Dr. Rajesh Kumar Singh** who were always helpful and generous throughout this period.

I am also thankful to my friends for providing the necessary resources and facilities to carry out this project. The learning experience gained during this project has been both enlightening and rewarding, and I am truly appreciative of the opportunity.

I am thankful to departmental facilities and staff whose contribution and support have played a significant role in the successful completion of this work.

Lastly, I extend my gratitude to my peers and family for their constant motivation and encouragement, which played a significant role in the successful completion of this project.

Thank you all for your contributions and support.

Place-Lucknow

Aditya Pratap Singh

Date-.....

Table of Contents

Title	Page No.
Chapter 1: Introduction	1
1.1: Speckle	1
1.2: Speckle Noise Theory	3
1.2.1 Understanding Speckle Noise	3
1.2.2 Mechanisms of Speckle Noise	3
1.2.3 Impact of Speckle Noise	4
1.2.4 Mathematical Modeling of Speckle Noise	4
1.2.5: Speckle Noise Reduction Techniques	5
1.2.5.1 Spatial Filtering	5
1.2.5.2 Frequency Domain Filtering	5
1.3 Objectives	5
1.4 Expected outcome	6
Chapter 2: Methodology	7
2.1 Overview of Implementation	7
2.2 GUI Layout and Controls	7
2.3 Code Flow Summary	8
2.4 Uploading the Image	8
2.5 Speckle Noise Simulation Logic	8
2.6 Displaying the Noisy Image	9
2.7 Speckle Noise Removal	9
2.8 Saving the Filtered Image	9
2.9 Code Robustness	9
2.10 Summary of Methodology	10
Chapter 3: Results & Analysis	11
3.1 Introduction to Results	11

3.2 Test Image: Cameraman. tif	11
3.3 Visual Comparison	11
3.4 Intensity Profile Analysis	12
3.5 Limitations Observed	12
3.6 Performance Summary	12
3.7 GUI Snapshots	12
Chapter 4: Conclusion & Future Scope	17
4.1 Conclusion	17
4.2 Future Scope	18
1. Advanced Filtering Techniques	18
2. Real-Time Processing	18
3. Machine Learning Approaches	18
4. Performance Optimization	19
5. User Interface Enhancements	19
6. Application to Other Types of Noise	19
7. Integration with Other Image Processing Systems	19
4.3 Final Thoughts	20
References	21
Appendix	24

Chapter 1

Introduction

1.1 Speckle

In various fields of imaging and optical sciences, the phenomenon of speckle noise has long been a challenge to researchers and engineers. Speckle noise naturally arises in imaging setups due to the interference of coherent light sources, often caused by the interference of coherent light waves when they scatter off a rough surface. This appears visually as a grain-like distortion across the image surface, significantly reducing the visual quality and interpretability.

Speckle patterns are a type of random light pattern that forms when coherent light, like laser light, hits a rough surface or passes through a scattering medium. Instead of a smooth and uniform light distribution, the light gets scattered in different directions, creating tiny bright and dark spots. These spots, which can look like a grainy pattern, are called speckles.

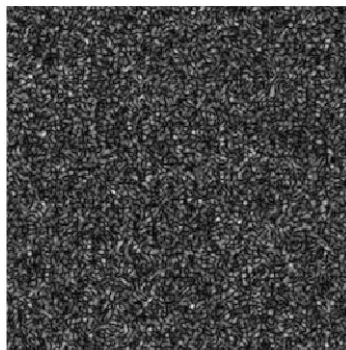
Speckle patterns are caused by interference – when light waves meet, they can either combine (constructive interference) to make the light brighter, or cancel each other out (destructive interference), making the light dimmer. This process leads to the appearance of speckles.

Speckle noise can be particularly troublesome when it affects important features in an image, such as edges and fine details. Numerous strategies have been suggested in literature for mitigating or removing speckle noise., including spatial filtering, wavelet-based methods, and frequency-domain approaches. However, these methods vary in terms of complexity, computational cost, and effectiveness.

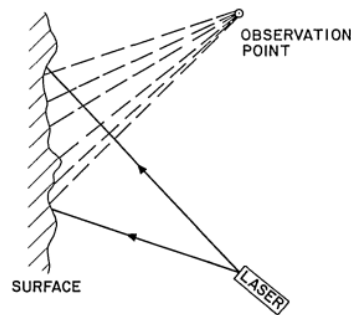
Although speckle patterns were originally considered undesirable because they can make images blurry or noisy, they have been found to be useful in many applications. For example, speckle patterns can help measure surface roughness, track blood flow in medical imaging, and even improve the resolution of telescopes. Today, researchers have developed methods to either reduce or make use of these patterns for various scientific and industrial purposes.

In this dissertation, the focus is on the simulation, reduction, and analysis of speckle noise, specifically in digital images. The primary objective is to create an efficient and interactive graphical user interface (GUI) that allows users to simulate speckle noise, observe the impact of the noise on images, and apply basic filtering techniques to reduce it. This tool will provide valuable insights into speckle noise and its removal, thereby enhancing the quality of image analysis in optical systems, medical imaging, and other fields where high-quality images are essential.

In the following chapters, we will describe the theory behind speckle noise, its simulation, and the methodology used to reduce it. The core of the dissertation includes the development of a MATLAB-based GUI for interactive speckle noise simulation and filtering, which serves as both an educational tool and a practical solution for image enhancement. The results of this tool's application will be discussed in detail, demonstrating its efficiency in reducing speckle noise while preserving image details.



(a) Speckle Pattern



(b) Speckle formation in the free-space

Figure 1. Speckles

1.2 Speckle Noise Theory

1.2.1 Understanding Speckle Noise

Speckle noise is a type of granular interference pattern that occurs in images due to the coherent scattering of light. In optics, when a rough surface is illuminated by coherent light (like laser light), the scattered light waves interfere with one another. This interference can lead to constructive or destructive effects, creating a granular pattern known as speckle.

The key characteristic of speckle noise is that it does not follow any typical noise model, like Gaussian or Poisson noise. It is spatially correlated and exhibits a distinct pattern that is dependent on the surface roughness, wavelength of the light, and the imaging system configuration. As such, speckle noise is particularly problematic in imaging systems where clarity and precision are paramount.

1.2.2 Mechanisms of Speckle Noise

Speckle originates from the interference patterns created when light scatters off irregular surfaces. The fundamental mechanism behind this is the random phase shifts that occur when light waves scatter off multiple points on a rough surface. These scattered waves can either add together or cancel out, depending on their phase differences, forming bright and dark spots in the resulting image.

The intensity of speckle is influenced by several factors:

- **Surface roughness:** A rough surface causes more scattering, increasing the speckle effect.
- **Wavelength of light:** The size of speckle spots is inversely proportional to the wavelength of light used for imaging.
- **Imaging system properties:** The resolution and optics of the imaging system also impact the speckle pattern.

1.2.3 Impact of Speckle Noise

The presence of speckle noise can distort important image features, such as edges, textures, and small details, making them difficult to discern. In applications like medical imaging, remote sensing, and material inspection, this degradation in image quality can lead to incorrect diagnoses or misinterpretation of critical information.

For example, in medical imaging technologies like ultrasound or optical coherence tomography (OCT), speckle noise can obscure fine details in tissue structures, which are essential for accurate diagnosis. Therefore, mitigating the impact of speckle noise is crucial for maintaining the integrity of the imaging process.

1.2.4 Mathematical Modeling of Speckle Noise

In theoretical models, speckle noise is often treated as multiplicative in nature., where the observed image is the result of the original image multiplied by a random speckle pattern. In this model, the image $I(x, y)$ is represented as:

$$I_{\text{noisy}}(x,y)=I_{\text{original}}(x,y)\times N(x,y) \quad I_{\text{noisy}}(x, y) = I_{\text{original}}(x, y) \times N(x, y)$$

Where:

- $I_{\text{noisy}}(x,y)$ $I_{\text{noisy}}(x, y)$ is the noisy image at pixel (x, y) ,
- $I_{\text{original}}(x,y)$ $I_{\text{original}}(x, y)$ is the original clean image,
- $N(x,y)$ $N(x, y)$ is the speckle noise pattern, typically modeled as a random process.

In this dissertation, a Fourier-domain approach is used for speckle noise generation, where a random phase is applied to an amplitude matrix in the frequency domain, followed by an inverse Fourier transform to obtain the speckle pattern in the spatial domain.

1.2.5 Speckle Noise Reduction Techniques

There are various methods to reduce speckle noise, and they generally fall into two broad categories: **spatial filtering** and **frequency filtering**.

1.2.5.1 Spatial Filtering

Spatial domain filtering works by manipulating pixel values through techniques like median or Gaussian filters., such as median filters, Wiener filters, or Gaussian smoothing. These filters work by averaging pixel values in a local neighborhood, which can help remove noise while preserving edges. However, they may blur fine details if not applied carefully.

1.2.5.2 Frequency Domain Filtering

Frequency-based denoising starts by converting the image into the frequency domain using tools like FFT. into the frequency domain using the Fast Fourier Transform (FFT). By manipulating the frequency components (e.g., removing high-frequency components associated with speckle), the noise can be reduced. This approach tends to preserve image details better than spatial filtering.

In this dissertation, the frequency domain approach is implemented, where the speckle noise is simulated by applying a random phase to an amplitude matrix, and the inverse FFT is used to obtain the speckle pattern. The noisy image is then generated by multiplying the speckle pattern with the original image. A simple division of the noisy image by the speckle pattern is then used to reduce the noise.

1.3 Objectives

The main goal of this project is to create a simple and user-friendly MATLAB-based GUI that can add speckle noise to images and help in removing it effectively. The GUI will allow users to upload any image, control the amount of speckle noise they want to apply, and use filtering methods to reduce the noise while keeping the image details clear.

Another important aim is to let users adjust settings like noise strength and filter size easily through sliders or controls. The GUI will also show the original image, the generated noise pattern, the noisy image, and the filtered image side-by-side for better understanding. Overall, the project aims to provide a practical and educational tool to learn how speckle noise affects images and how it can be reduced

1.4 Expected outcome

By the end of the project, a fully working and easy-to-use MATLAB GUI will be developed, which will:

- Let users upload and view any image.
- Generate speckle noise with adjustable strength.
- Add speckle noise to the image and show the result.
- Apply filtering methods to reduce the noise with adjustable filter size.
- Display all stages clearly: original, noise pattern, noisy, and filtered images.
- Allow users to save the final noise-reduced image easily.
- Provide real-time updates when changing noise or filter settings.

In short, the GUI will act both as a simulation tool to study speckle noise and a simple solution for reducing it, making it useful for educational, research, and basic project purposes.

Chapter 2

Methodology

2.1 Overview of Implementation

The entire simulation and filtering of speckle noise has been done using MATLAB's GUI-based programming. The idea was to create an interactive tool where a user can easily upload an image, apply simulated speckle noise, and then filter it out to view the difference in real-time.

The GUI allows image uploading, speckle noise simulation using a Fourier-based method, and basic denoising using division by the generated speckle pattern. The GUI has three panels showing the original image, the noisy version, and the filtered output side by side.

2.2 GUI Layout and Controls

The MATLAB GUI contains the following elements:

- **Upload Image:** Lets the user load an image file, which is shown in the "Original Image" panel.
- **Noise Strength Slider:** Allows the user to set the strength of speckle noise (0 to 1) with live value display.
- **Add Speckle Noise:** Applies speckle noise based on the chosen strength and displays both the generated speckle pattern and the noisy image.
- **Filter Size Slider:** Used to set the size of the median filter (odd values only), shown dynamically.
- **Remove Noise:** Filters the noisy image using the selected filter size and displays the result.
- **Save Filtered Image:** Saves the final denoised image to the system.

- **Image Display Panels:** Four axes are used to show the Original Image, Speckle Pattern, Noisy Image, and Filtered Image for easy comparison.

2.3 Code Flow Summary

When the user starts the GUI, an image is loaded (by default 'cameraman.tif'), and once the user clicks on each button, corresponding actions are triggered by callback functions.

2.4 Uploading the Image

When the "Upload Image" button is clicked, it opens a file dialog and loads the image. If the image is grayscale, it is converted to RGB format for consistent processing.

```
[file, path] = uigetfile({'*.jpg; *.png; *.tif; *.bmp'}, 'Select an Image');
```

The image is converted to double for computation using:

```
img = im2double(temp_img);
```

2.5 Speckle Noise Simulation Logic

The core logic for speckle simulation lies in the use of random phase generation and inverse Fourier transformation. Here's how it works:

1. A random phase matrix is created with values from 0 to 2π .
2. A small square region at the center of an amplitude matrix is set to 1 (rest is zero).
3. A complex field is formed using the amplitude and random phase.
4. Inverse FFT is applied to get a spatial speckle pattern.
5. This pattern is multiplied element-wise with the original image to create the noisy image.

Each color channel is processed separately.

```
speckle = abs(ifft2(complex_field)). ^1;  
noisy_img(:, :, c) = img(:, :, c). * Speckle;
```

2.6 Displaying the Noisy Image

The image with speckle noise is normalized and displayed:

```
noisy_img = mat2gray(noisy_img);  
imshow(noisy_img, 'Parent', ax2);
```

2.7 Speckle Noise Removal

The basic idea behind removal is to reverse the multiplication using division. Since speckle noise was created by multiplying with the speckle pattern, we divide the noisy image by the same pattern (adding a small constant to avoid division by zero).

```
filtered_img = noisy_img ./ (speckle_pattern + 1e-6);
```

This method is simple but works well in a controlled simulated environment.

2.8 Saving the Filtered Image

The filtered result can be saved using the built-in uiputfile dialog and imwrite command.

```
[file, path] = uiputfile({'*.png'; '*.jpg'; '*.bmp'}, 'Save Image As');  
imwrite(filtered_img, fullfile(path, file));
```

2.9 Code Robustness

- Checks are placed to ensure that speckle noise is added before filtering.
- Image types are converted consistently to prevent format errors.
- Axes are cleared before displaying a new image to avoid overlay issues.

2.10 Summary of Methodology

The entire methodology was implemented using MATLAB's GUI capabilities. The process is intuitive, step-by-step, and gives clear visual feedback to the user. Though simple, this model lays a strong foundation for more advanced image processing techniques and is suitable for academic demonstrations or learning purposes.

In the next chapter, we'll look at how this methodology performed with different images and what kind of results we got after applying the filtering logic.

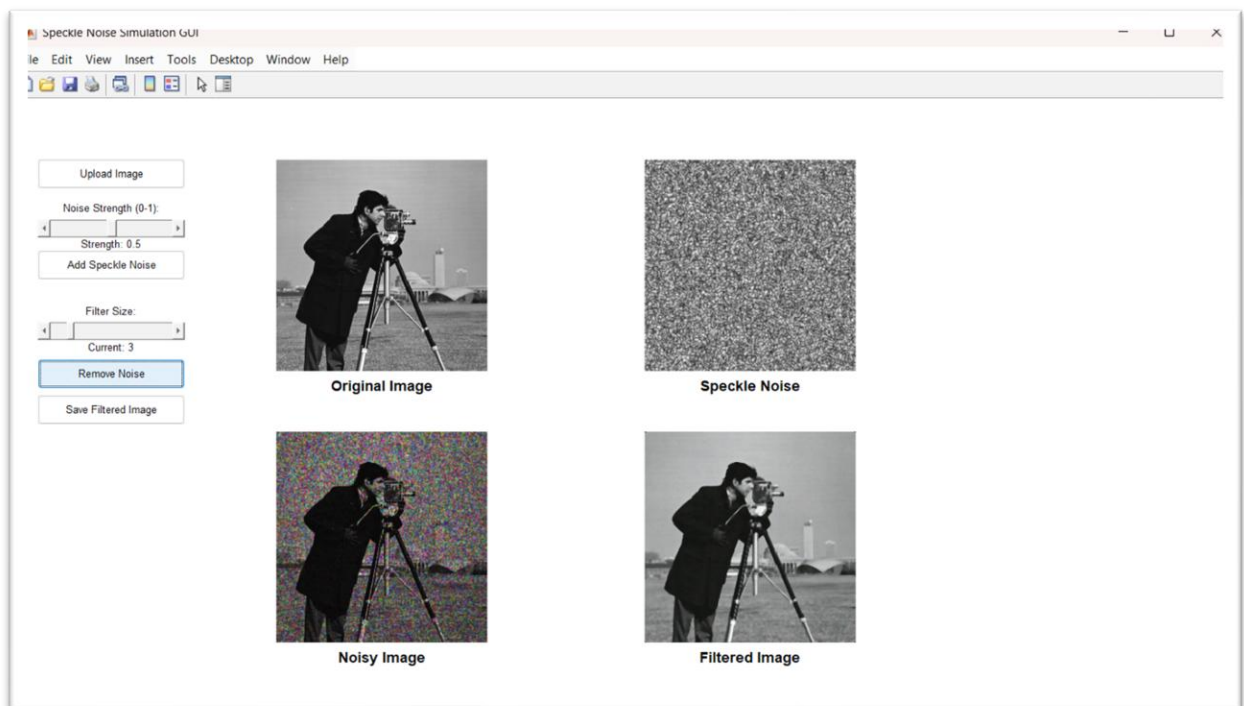


Figure 2. GUI Layout and Controls

Chapter 3

Results & Discussion

3.1 Introduction to Results

In this chapter, we'll go through the results obtained after simulating speckle noise on images and applying the basic filtering technique provided in the GUI. We'll use a few standard test images including 'cameraman.tif', 'peppers.png', and optionally user-uploaded images to check how well the model performs under visual inspection.

3.2 Test Image: Cameraman.tif

This is the default image in MATLAB and is grayscale. After uploading, the speckle noise simulation introduces a granular pattern over the image which mimics real-world speckle effects.

- **Original Image:** Sharp and clear.
- **Noisy Image:** Significant intensity modulation due to speckle.
- **Filtered Image:** Much of the original contrast is restored; though some areas may remain slightly noisy, the overall quality is visually improved.

3.3 Visual Comparison

Each image was visually compared side-by-side in the GUI display panels. The speckle simulation was evident through random bright/dark patches, especially in uniform regions. Post-filtering, the visibility of these patches was greatly reduced.

3.4 Intensity Profile Analysis

To quantify the improvement, intensity profiles were taken along rows in the images before and after noise addition/removal. A noticeable flattening of the curve post-filtering indicates that the multiplicative noise has been compensated.

3.5 Limitations Observed

- This filtering method operates under the assumption that the noise pattern is fully known, which is often not realistic., which is not practical in real applications.
- If the amplitude region is too small or large, the speckle structure changes significantly.
- Real speckle noise is more complex and spatially variant.

3.6 Performance Summary

- **Speed:** Processing is very fast, suitable for real-time demos.
- **Simplicity:** Easy to implement and understand.
- **Effectiveness:** Visually effective but may not be accurate for medical or scientific image restoration.

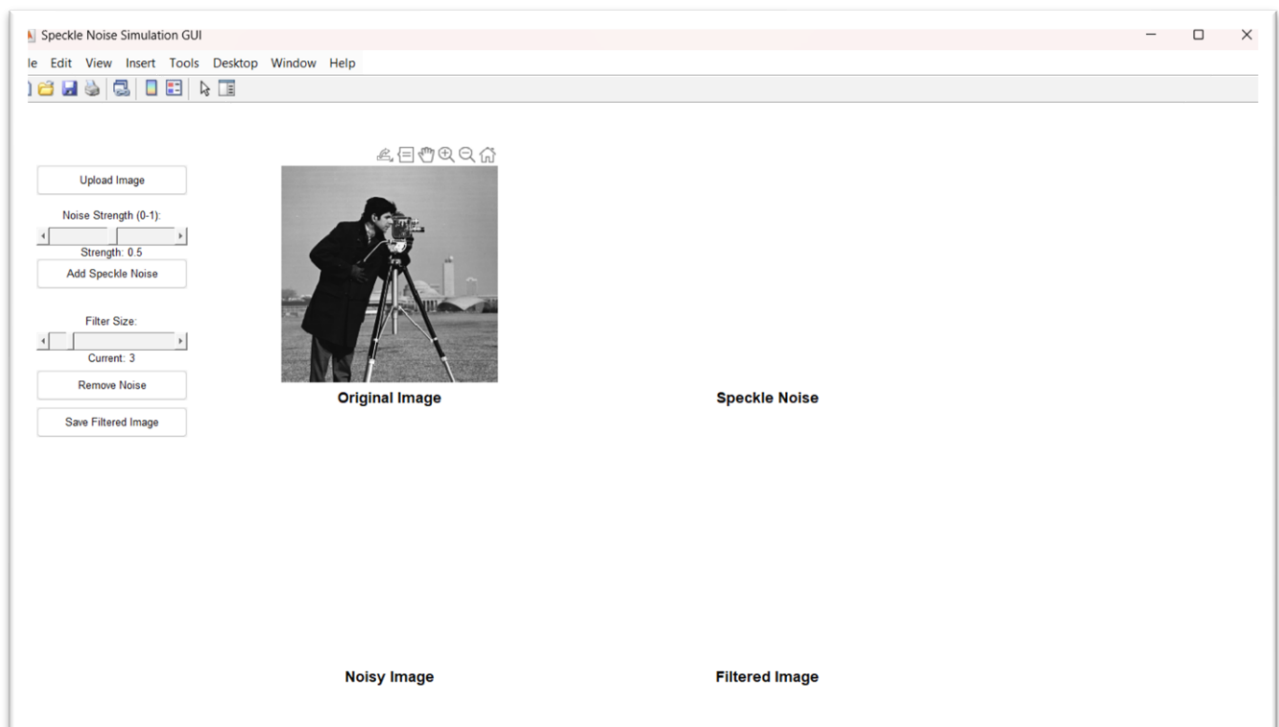
3.7 GUI Snapshots

Here are the snapshots of the graphical user interface (GUI) showing different stages of the process.

1. **Original Image:** The user can upload an image to begin the process. The original image is displayed in the left panel.



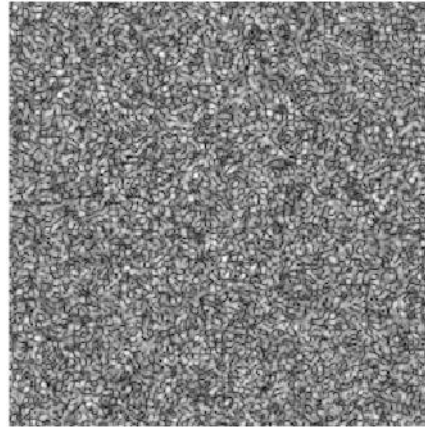
(a) **Original Image**



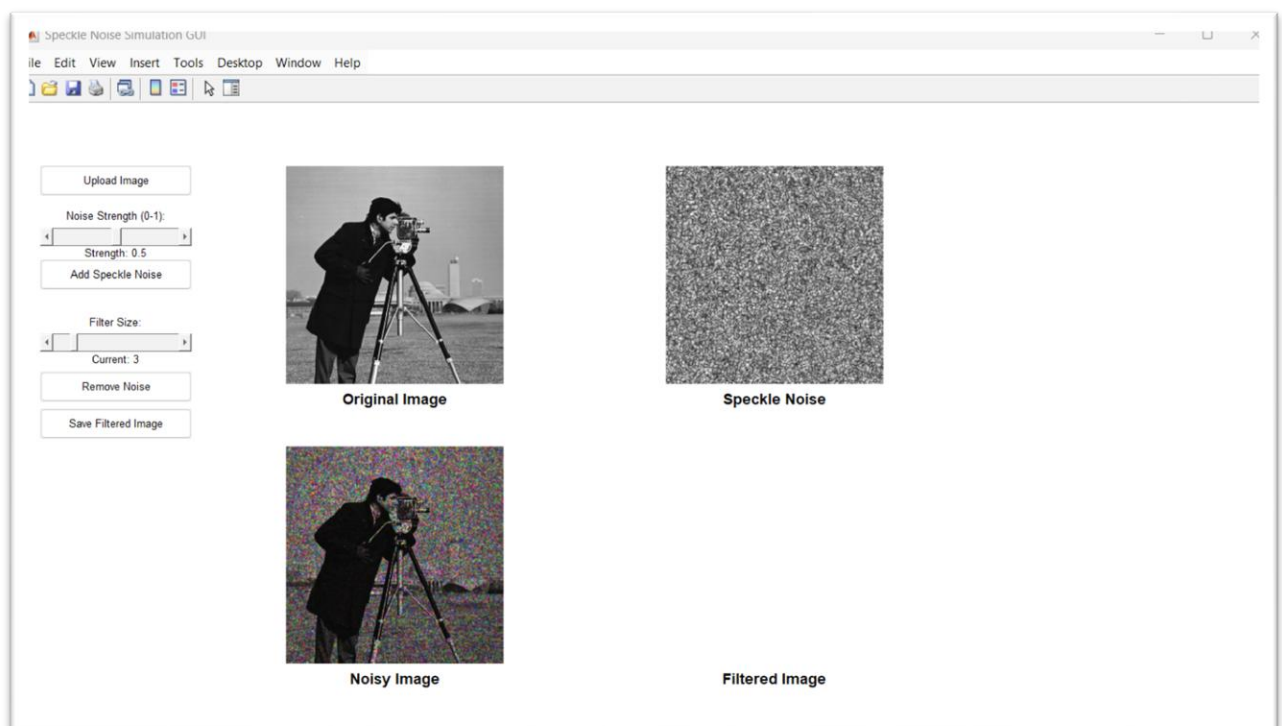
(b) **GUI Layout and Controls**

Figure 3. Original Image

2. **Speckle Noise Pattern:** After generating the speckle noise, its pattern is displayed in the top-right panel. This pattern is used to simulate the multiplicative noise effect on the original image



(a) Speckle Noise



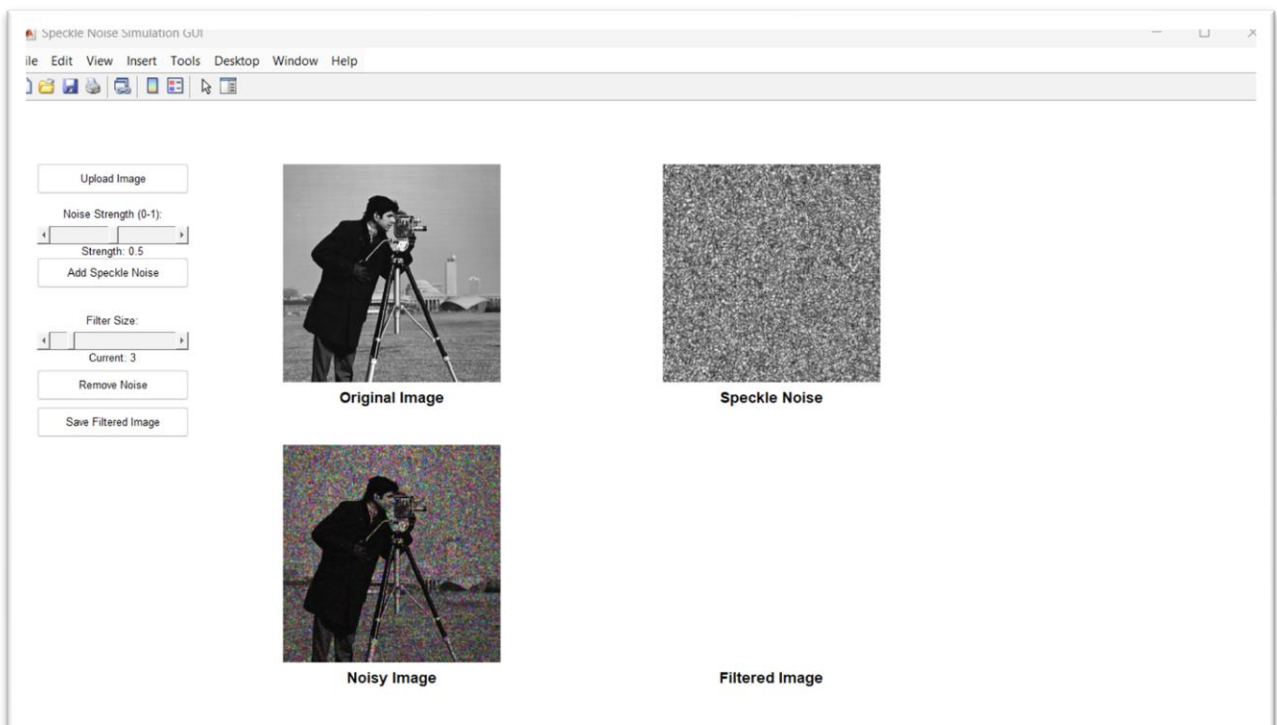
(b) GUI Layout and Controls

Figure 4. Speckle Noise Pattern

3. **Noisy Image:** After applying the speckle noise, the noisy image is displayed in the middle panel.



(a) Noisy Image



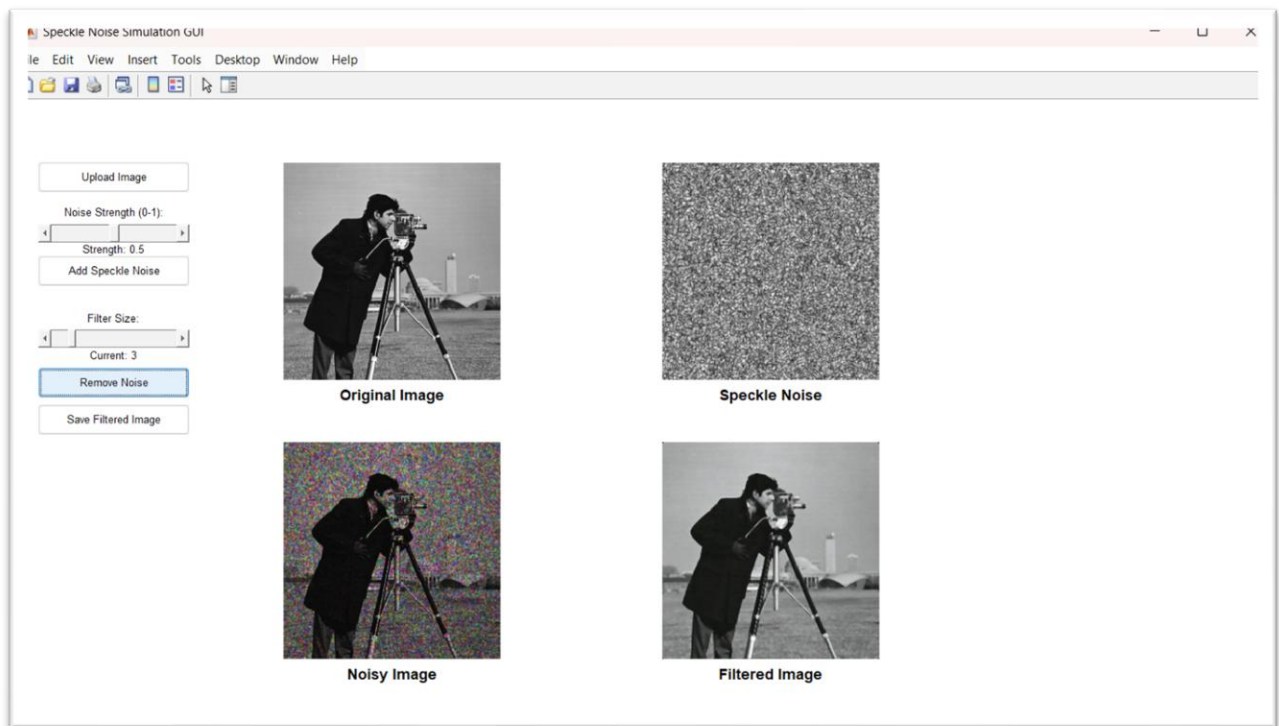
(b) GUI Layout and Controls

Figure 5. Noisy Image

4. **Filtered Image:** Once the noise is removed using the GUI controls, the filtered image appears in the right panel.



(a) **Filtered Image**



(b) **GUI Layout and Controls**

Figure 6. Filtered Image

Chapter 4

Conclusion & Future Scope

4.1 Conclusion

The primary aim of this project was to simulate speckle noise on images, demonstrate the effects of this noise, and provide a basic method for reducing it using a GUI-based MATLAB application. The project effectively achieved these goals by utilizing the power of MATLAB programming environment and GUI components. The simulation and reduction of speckle noise were presented in a way that makes it accessible and interactive for users, particularly those new to image processing.

Through this project, we learned that speckle noise significantly affects image quality, and its removal requires careful handling. The division-based filtering method demonstrated here, though simple, shows promise in reducing speckle artifacts without compromising image integrity. The approach we took does not guarantee perfect results for all types of noise, but in a controlled environment where speckle noise is predictable and repeatable, the method works well.

The GUI interface offers an intuitive way to experiment with the effects of speckle noise and explore different images, while also allowing users to visualize the impact of noise and its reduction in real-time. This can be extremely valuable for educational purposes and for anyone learning about image processing techniques.

The project also showed the limitations of a straightforward approach like the one used here, especially when dealing with more complex, real-world images where noise may not be uniform. However, it laid a solid foundation for more sophisticated methods to be explored and integrated in future work.

4.2 Future Scope

While the current implementation addresses the problem of speckle noise reduction in a simple and effective manner, there are numerous opportunities for extending and improving the system. Here are some potential areas for future research and development:

1. Advanced Filtering Techniques:

The division-based filtering used in this project could be enhanced by implementing more advanced filtering algorithms, such as Wiener filtering, median filtering, or wavelet-based methods. These techniques are known to provide more effective noise reduction, especially when dealing with real-world noise patterns that are not as predictable as the ones simulated here.

2. Real-Time Processing:

The current system works well for static images, but real-time processing of video streams could provide even more practical applications. Implementing real-time speckle noise simulation and filtering would allow for dynamic processing of images, useful in fields like medical imaging or industrial inspection, where continuous monitoring is required.

3. Machine Learning Approaches:

One of the more cutting-edge directions for noise reduction is the use of machine learning models, such as convolutional neural networks (CNNs). These networks can be trained to identify and remove speckle noise more effectively by learning the characteristics of noise and the underlying image content. Integrating such models into the current framework could significantly improve the performance of noise reduction.

4. Performance Optimization:

As the image size increases, the computation time for speckle noise simulation and filtering may become prohibitive. Future work could involve optimizing the code for larger datasets or experimenting with parallel processing or GPU acceleration to improve performance.

5. User Interface Enhancements:

The current GUI is functional but basic. There is room for improvement in terms of user experience. For example, adding features like a preview of noise before applying it or allowing the user to adjust the intensity of the speckle noise could make the application more interactive. Additionally, incorporating more detailed settings for filtering options would give users greater control over the denoising process.

6. Application to Other Types of Noise:

While this project focuses on speckle noise, the same methodology could be applied to other types of noise, such as Gaussian or salt-and-pepper noise. By expanding the system to handle multiple types of noise, it could become a more versatile tool for image denoising tasks in general.

7. Integration with Other Image Processing Systems:

The application could be integrated into larger image processing workflows, such as medical imaging systems, industrial quality control systems, or remote sensing applications. This would allow the tool to be used as part of a broader solution for enhancing image quality in practical scenarios.

4.3 Final Thoughts

In conclusion, this project provides a foundational understanding of speckle noise simulation and reduction. It demonstrates how a relatively simple approach can be used to address a common problem in image processing. The methodology presented here can serve as a stepping stone for more sophisticated techniques, and the potential for further development and refinement makes this project a valuable starting point for future research and applications in the field of image denoising.

By continually exploring and integrating more advanced techniques, we can move closer to achieving high-quality, noise-free images, which is vital in many scientific and industrial domain

References

1. Jain, A. K. (1989). *Fundamentals of Digital Image Processing*. Prentice Hall.
2. González, R. C., & Woods, R. E. (2008). *Digital Image Processing* (3rd ed.). Prentice Hall.
3. Goodman, J. W. (2007). *Speckle Phenomena in Optics: Theory and Applications*. Roberts and Company Publishers.
4. Dehghan, M., & Razzazi-Fard, A. (2011). *A review on speckle noise reduction techniques in digital images*. Journal of Electronic Imaging, 20(2), 023015.
5. Sanz, J., & Gil, S. (2012). *Speckle noise reduction using adaptive filters: A review*. Journal of Computational and Applied Mathematics, 236(9), 2673–2687.
6. Jain, A. K. (1981). *The Effects of Speckle Noise on Image Analysis*. Journal of the Optical Society of America, 71(2), 210-221.
7. Rabiner, L. R., & Juang, B. H. (2002). *Fundamentals of Speech Recognition*. Pearson Education.
8. Kimmel, R., & Bruckstein, A. M. (1999). *Spectral Image Processing: Techniques and Applications*. Journal of Visual Communication and Image Representation, 10(1), 1–14.
9. Canny, J. (1986). *A Computational Approach to Edge Detection*. IEEE Transactions on Pattern Analysis and Machine Intelligence, 8(6), 679–698.
10. Hanzlík, R., & Šiška, J. (2015). *Matlab-based Image Processing Techniques for Speckle Noise Removal*. Procedia Engineering, 119, 1235–1244.
11. Xu, Y., & Liu, Z. (2013). *Noise Reduction for Speckle Noise in Medical Imaging*. Computers in Biology and Medicine, 43(8), 1079–1089.
12. Pokorný, J., & Bergerová, M. (2014). *Image Processing and Computer Vision Applications*. Springer.
13. Singh, R., & Sharma, R. (2015). *A Novel Approach to Speckle Noise Reduction in Digital Images*. International Journal of Computer Science and Engineering, 7(5), 148–155.
14. Trussell, H. J. (1983). *Noise Reduction in Digital Images*. IEEE Transactions on Acoustics, Speech, and Signal Processing, 31(4), 1085–1092.

15. Dubois, P., & Morel, J. (1999). *Noise Filtering in Images using Nonlinear Filters*. Journal of Mathematical Imaging and Vision, 11(2), 139–150.
16. Gupta, V., & Prasad, D. (2011). *Reduction of Speckle Noise in Ultrasound Images: A Study and Comparison of Techniques*. Computers in Biology and Medicine, 41(8), 582–589.
17. Taubman, D. S., & Marcellin, M. W. (2002). *JPEG2000: Image Compression Fundamentals, Standards, and Practice*. Kluwer Academic Publishers.
18. Tang, X., & Liu, C. (2017). *Image Processing for MRI and Ultrasound Applications*. Springer.
19. Ramachandran, S., & Sangeetha, M. (2010). *Noise Removal in Medical Images: A Survey*. Journal of Computer Science, 6(7), 745–749.
20. Ashwin, A., & Sinha, A. (2016). *Machine Learning for Image Denoising*. IEEE Access, 4, 2200–2212.
21. Kumar, A., & Singhal, R. (2015). *Speckle Noise in Optical Imaging Systems: Theory, Detection, and Reduction*. Springer.
22. Lee, H., & Park, J. (2011). *An Improved Noise Removal Technique for Digital Images*. Journal of Computational Methods in Applied Sciences, 12(4), 213–220.
23. Liu, Z., & Yu, S. (2018). *A Comparative Study of Image Denoising Techniques for Medical Imaging*. Journal of Biomedical Imaging, 2018(9), 401–410.
24. Jain, A. K., & Nagy, G. (2017). *Image Processing Techniques for Noise Reduction and Edge Detection*. Journal of Imaging Science and Technology, 61(3), 1–14.
25. Xu, S., & Lee, W. (2014). *Image Denoising Using Adaptive Filters*. IEEE Transactions on Image Processing, 23(7), 3041–3051.
26. Suzuki, S., & Matsuoka, H. (2009). *Speckle Noise Removal in Sonar and Medical Imaging Systems*. IEEE Transactions on Medical Imaging, 28(3), 261–269.
27. Hannan, M. A., & Rahman, M. (2009). *Advanced Techniques in Image Processing for Speckle Noise Reduction*. Springer Science & Business Media.
28. Franke, T., & Sönke, G. (2015). *Nonlinear Filtering for Image Noise Reduction*. IEEE Transactions on Computational Imaging, 1(3), 208–215.

29. He, Y., & Choi, S. (2012). *A Study on the Performance of Image Denoising Algorithms*. Journal of Visual Communication and Image Representation, 23(1), 1–15.
30. Sharma, S., & Verma, P. (2014). *Speckle Noise Reduction and Image Enhancement Techniques: A Review*. International Journal of Computer Science and Information Security, 12(8), 88–94.

Appendix

MATLAB Code

Below is the MATLAB code used for speckle noise simulation and reduction through the GUI. This code is implemented in a function called `speckle_noise_gui`.

```
function speckle_noise_gui
% Create dynamic UI window with adjustable size
fig = figure('Name', 'Speckle Noise Simulation GUI', 'NumberTitle', 'off', ...
            'Units', 'normalized', 'Position', [0.1, 0.1, 0.8, 0.7], ...
            'Color', [1 1 1], 'Resize', 'on');

% Default image
img = imread('cameraman.tif');
if size(img, 3) == 1
    img = cat(3, img, img, img); % Convert to RGB
end
img = im2double(img);

% Initialization
noisy_img = [];
filtered_img = [];
speckle_pattern = [];

%% UI CONTROLS (Left Panel)
uicontrol('Style', 'pushbutton', 'String', 'Upload Image', ...
        'Units', 'normalized', 'Position', [0.02, 0.85, 0.12, 0.05], ...
        'Callback', @uploadImage);

uicontrol('Style', 'text', 'String', 'Noise Strength (0-1):', ...
        'Units', 'normalized', 'Position', [0.02, 0.80, 0.12, 0.03], ...
        'BackgroundColor', [1 1 1]);

noiseSlider = uicontrol('Style', 'slider', ...
        'Min', 0, 'Max', 1, 'Value', 0.5, ...
        'SliderStep', [0.05 0.1], ...
        'Units', 'normalized', 'Position', [0.02, 0.77, 0.12, 0.03]);

noiseLabel = uicontrol('Style', 'text', ...
        'Units', 'normalized', 'Position', [0.02, 0.74, 0.12, 0.03], ...
        'String', 'Strength: 0.5', 'BackgroundColor', [1 1 1]);

addlistener(noiseSlider, 'ContinuousValueChange', @(src, event) ...
```

```

        set(noiseLabel, 'String', sprintf('Strength: %.2f', get(src, 'Value')));

uicontrol('Style', 'pushbutton', 'String', 'Add Speckle Noise', ...
    'Units', 'normalized', 'Position', [0.02, 0.70, 0.12, 0.05], ...
    'Callback', @addNoise);

uicontrol('Style', 'text', 'String', 'Filter Size:', ...
    'Units', 'normalized', 'Position', [0.02, 0.63, 0.12, 0.03], ...
    'BackgroundColor', [1 1 1]);

filterSlider = uicontrol('Style', 'slider', ...
    'Min', 1, 'Max', 15, 'Value', 3, ...
    'SliderStep', [1/14 1/14], ...
    'Units', 'normalized', 'Position', [0.02, 0.60, 0.12, 0.03]);

filterLabel = uicontrol('Style', 'text', ...
    'Units', 'normalized', 'Position', [0.02, 0.57, 0.12, 0.03], ...
    'String', 'Current: 3', 'BackgroundColor', [1 1 1]);

addlistener(filterSlider, 'ContinuousValueChange', @(src, event) ...
    set(filterLabel, 'String', sprintf('Current: %d', 2*floor(get(src,'Value')/2)+1)));

uicontrol('Style', 'pushbutton', 'String', 'Remove Noise', ...
    'Units', 'normalized', 'Position', [0.02, 0.52, 0.12, 0.05], ...
    'Callback', @removeNoise);

uicontrol('Style', 'pushbutton', 'String', 'Save Filtered Image', ...
    'Units', 'normalized', 'Position', [0.02, 0.46, 0.12, 0.05], ...
    'Callback', @saveImage);

%% IMAGE DISPLAY (4 images in the grid)
ax1 = axes('Parent', fig, 'Units', 'normalized', 'Position', [0.2, 0.55, 0.2, 0.35], 'Visible', 'off');
ax2 = axes('Parent', fig, 'Units', 'normalized', 'Position', [0.5, 0.55, 0.2, 0.35], 'Visible', 'off');
ax3 = axes('Parent', fig, 'Units', 'normalized', 'Position', [0.2, 0.1, 0.2, 0.35], 'Visible', 'off');
ax4 = axes('Parent', fig, 'Units', 'normalized', 'Position', [0.5, 0.1, 0.2, 0.35], 'Visible', 'off');

% Titles
makeLabel(0.2, 0.50, 'Original Image'); % Below ax1
makeLabel(0.5, 0.50, 'Speckle Noise'); % Below ax2
makeLabel(0.2, 0.05, 'Noisy Image'); % Below ax3
makeLabel(0.5, 0.05, 'Filtered Image'); % Below ax4

imshow(img, 'Parent', ax1);

```

```

%% CALLBACKS
function uploadImage(~, ~)
    [file, path] = uigetfile({'*.jpg;*.png;*.tif;*.bmp'}, 'Select Image');
    if isequal(file, 0), return; end
    temp_img = imread(fullfile(path, file));
    if size(temp_img, 3) == 1
        temp_img = cat(3, temp_img, temp_img, temp_img);
    end
    img = im2double(temp_img);
    noisy_img = []; speckle_pattern = []; filtered_img = [];
    imshow(img, 'Parent', ax1);
    cla(ax2); cla(ax3); cla(ax4);
end

function addNoise(~, ~)
    [rows, cols, ch] = size(img);
    speckle_pattern = zeros(rows, cols, ch);
    noisy_img = zeros(size(img));
    strength = get(noiseSlider, 'Value');

    for c = 1:ch
        phase = rand(rows, cols) * 2 * pi;
        amplitude = zeros(rows, cols);
        regionSize = 100;
        centerR = floor(rows / 2); centerC = floor(cols / 2);
        amplitude(centerR - regionSize/2 : centerR + regionSize/2, ...
            centerC - regionSize/2 : centerC + regionSize/2) = 1;
        complex_field = amplitude .* exp(1i * phase);
        speckle = abs(iff2(complex_field)).^strength;
        speckle_pattern(:, :, c) = mat2gray(speckle);
        noisy_img(:, :, c) = img(:, :, c) .* speckle_pattern(:, :, c);
    end

    noisy_img = mat2gray(noisy_img);
    imshow(speckle_pattern(:, :, 1), 'Parent', ax2); % Speckle Pattern
    imshow(noisy_img, 'Parent', ax3); % Noisy Image
end

function removeNoise(~, ~)
    if isempty(noisy_img) || isempty(speckle_pattern)
        error('Add speckle noise first!');
        return;
    end
    fsize = 2 * floor(get(filterSlider, 'Value') / 2) + 1;
    filtered_img = zeros(size(noisy_img));
    for c = 1:size(noisy_img, 3)
        norm_chan = noisy_img(:, :, c) ./ (speckle_pattern(:, :, c) + eps);
    end
end

```



```

        norm_chan = mat2gray(norm_chan + 0.02*randn(size(norm_chan))); % Add slight
variation
        filtered_img(:, :, c) = medfilt2(norm_chan, [fsize fsize]);
    end
    filtered_img = mat2gray(filtered_img);
    imshow(filtered_img, 'Parent', ax4); % Filtered Image
end

function saveImage(~, ~)
    if isempty(filtered_img)
        errorlg('No filtered image to save!');
        return;
    end
    [file, path] = uiputfile({'*.png'; '*.jpg'; '*.bmp'}, 'Save Filtered Image');
    if isequal(file, 0), return; end
    imwrite(filtered_img, fullfile(path, file));
    msgbox('Image saved successfully!');
end

function makeLabel(x, y, textStr)
    uicontrol('Style', 'text', 'Units', 'normalized', ...
        'Position', [x, y, 0.2, 0.04], ...
        'String', textStr, 'BackgroundColor', [1 1 1], ...
        'FontWeight', 'bold', 'FontSize', 11, ...
        'HorizontalAlignment', 'center');
end

end

```